

Task: Secure Coding Review Submitted by: Maryam Siddique Date: 8-15-2025

Description: This document presents a secure coding review of a sample Python application, highlighting vulnerabilities, suggested remediations, and best practices for secure development.

Sample Insecure Python Code:

```
# Sample vulnerable code
import sqlite3

# Vulnerable SQL query (SQL Injection possible)
def get_user(username):
    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE username='" + username + "'"
    cursor.execute(query)
    result = cursor.fetchall()
    conn.close()
    return result
```

Identified Vulnerabilities:

1. SQL Injection: Direct string formatting allows malicious input to modify queries.
2. Lack of input validation/sanitization.
3. No error handling or logging.

Remediation Steps:

1. Use parameterized queries to prevent SQL injection:

```
cursor.execute("SELECT * FROM users WHERE username=?", (username,))
```

1. Validate and sanitize all user inputs.
2. Add error handling and logging:

```
try:
    # execute query
except Exception as e:
    print(f"Error: {e}")
```

Best Practices:

- Avoid hardcoded credentials and secrets.
- Keep dependencies updated; scan for known vulnerabilities.

- Implement proper authentication and authorization.
- Use secure cryptography standards (bcrypt, AES).
- Follow least privilege principle for access control.
- Document security decisions and perform regular code reviews.

Tools Used:

- Bandit (Python static analyzer)
- Semgrep (cross-language security rules)
- Manual code inspection

Conclusion: This review identifies critical security issues and provides actionable recommendations. Applying these fixes will significantly improve application security and align with secure coding standards.